Trasladar una aplicación web desde una máquina Windows o Mac (ambas en el host) hacia una máquina virtual Ubuntu Server 24.04 LTS mediante la tecnología Docker.

Objetivo:

Crear una imagen que contenga una página web y el servidor Nginx para ejecutarla en la computadora host del estudiante, guardarla en un archivo .tar y trasladarla a una máquina virtual Ubuntu, donde funcionará sin reconstruirla ni instalar Nginx por separado.

Propósito de la práctica:

- Distinguir código fuente, Dockerfile, imagen, contenedor y máquina virtual.
- Construir una imagen y publicar un puerto para acceder desde el navegador.
- Transportar la misma imagen mediante docker save y docker load.
- Comprobar el despliegue y detener los recursos al terminar.

Flujo de la práctica

Windows o Mac → construir imagen → ejecutar y probar → guardar TAR → copiar a Ubuntu → cargar imagen → ejecutar y probar desde el navegador.

Elemento	Qué significa en esta práctica
Código fuente	index.html contiene la estructura de la página y su JavaScript.
Dockerfile	Secuencia de instrucciones para indicar qué base usar y qué archivos incorporar.
Imagen	Paquete con Nginx y la página. Es lo que se transporta.
Contenedor	Instancia de esa imagen que Docker ejecuta.
Máquina virtual	Ubuntu tiene su propio sistema operativo y kernel.

Docker Desktop proporciona un entorno Linux en Windows y Mac. En Ubuntu, Docker Engine usa el kernel Linux de la máquina virtual. No se ésta convirtiendo una aplicación de Windows en una de Linux: se está construyendo y ejecutando una imagen Linux en ambos entornos.

Se debe comprobar la versión y arquitectura del host y ubuntu antes de iniciar. Ej. En Mac arm64 y Ubuntu arm64.

1. Preparar los equipos

Dónde se ejecuta cada instrucción

Etiqueta	Lugar y herramienta
WINDOWS	PowerShell en la computadora física. Docker Desktop abierto, con WSL 2 y contenedores Linux.
MAC	Terminal de macOS en la computadora física. Docker Desktop abierto.
UBUNTU	Consola de Ubuntu Server dentro de VMware, o una sesión SSH hacia esa VM.
WINDOWS Y MAC	El mismo comando funciona en PowerShell y Terminal, salvo indicación expresa.

Instalar Docker Desktop siguiendo el enlace correspondiente de la sección 10. En Windows, si WSL no está instalado, abrir PowerShell como administrador, ejecutar wsl --install y reiniciar si se solicita. Después instalar Docker Desktop con el backend WSL 2. En Mac eligir el instalador para Apple Silicon o Intel según tu equipo.

Comprobar Docker en Windows o Mac

```
docker version
docker info --format '{{.OSType}}/{{.Architecture}}'
```

Debe verse Client y Server; el tipo de sistema de Docker debe ser linux. Si no conecta con el motor, abrir Docker Desktop y esperar a que esté listo.

Comprobar Ubuntu en la máquina virtual

```
cat /etc/os-release
uname -m
dpkg --print-architecture
```

Procesador	uname o Docker info	Nombre para Docker y apt
Intel o AMD de 64 bits	x86_64	amd64
ARM de 64 bits	aarch64 o arm64	arm64

La ruta principal requiere la misma arquitectura en origen y destino. Una VM ARM de un Mac Apple Silicon no se ejecuta mediante virtualización nativa en una PC Intel/AMD.

La VM debe tener internet y una IP accesible desde el anfitrión. En VMware puede utilizarse una red puente o NAT que permita comunicación anfitrión–invitado. Si se clonan varias VM en una red, cada una debe tener su propia IP. Los comandos no requieren instalar Docker Desktop dentro de Ubuntu.

2. Crear los archivos de la aplicación

WINDOWS Y MAC. Crea una carpeta llamada practica-docker y ábrela en un editor de texto, por ejemplo Visual Studio Code. Guarda los archivos como texto UTF-8. No uses Word para guardar el código.

Archivo index html

Crea un archivo llamado exactamente index.html. Este ejemplo compacto permite realizar la práctica sin descargar archivos adicionales:

```
<!doctype html>
<html lang="es">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Mi primera aplicación con Docker</title>
</head>
<body>
  <h1>Mi aplicación viajó a Ubuntu</h1>
  <p>Equipo: escribe aquí los nombres de los integrantes</p>
  <p>Versión 1.0</p>
  <button id="boton">Probar aplicación</button>
  <p id="salida" role="status">Clics: 0</p>
  <script>
    let clics = 0;
    document.querySelector('#boton').onclick = () => {
      clics++;
      document.querySelector('#salida').textContent = 'Clics: ' + clics;
    };
  </script>
</body>
</html>
```

El HTML se sirve desde Nginx, pero el JavaScript del contador se ejecuta en el navegador. El contador se reinicia al recargar: no hay base de datos ni almacenamiento persistente.

Archivo Dockerfile

```
FROM nginx:stable-alpine
COPY index.html /usr/share/nginx/html/index.html
EXPOSE 80
```

FROM selecciona una imagen base con Nginx sobre Alpine Linux. COPY incorporar la página en la imagen. EXPOSE documenta el puerto 80; por sí solo no lo publica. El comando de inicio de Nginx se hereda de la imagen base.

Archivo punto dockerignore

Guardar el siguiente contenido en un archivo llamado .dockerignore. Así el TAR y otros archivos no se envían como contexto en futuras construcciones.

```
*
!Dockerfile
!index.html
```

En Windows activar la visualización de extensiones: Dockerfile no debe terminar en .txt, seleccionar texto sin formato antes de guardar.

3. Construir y probar en Windows o Mac

Abrir la terminal en la carpeta del proyecto

WINDOWS. En el Explorador abrir practica-docker; escribir powershell en la barra de direcciones y pulsar Enter. Comprobar los archivos con:

```
Get-Location
Get-ChildItem -Force
```

MAC. En Terminal escribir cd seguido de un espacio, arrastrar la carpeta practica-docker desde Finder a la terminal y pulsar Enter. Comprobar:

```
pwd
ls -la
```

Los comandos siguientes son iguales en WINDOWS Y MAC. Se deb permanecer en la carpeta que contiene Dockerfile e index.html.

Construir la imagen

```
docker build -t aula-web:1.0 .
```

El punto final significa “usa esta carpeta como contexto”. -t asigna el nombre aula-web y la etiqueta 1.0. En la primera construcción se descarga Nginx, por lo que se require de internet. La imagen queda en el almacenamiento de Docker, no como un archivo visible en la carpeta.

Crear y arrancar el contenedor

```
docker run -d --name web-aula -p 127.0.0.1:8080:80 aula-web:1.0
docker ps
```

-d ejecuta en segundo plano; --name asigna un nombre al contenedor. El puerto 8080 de la computadora host se conecta con el 80 de Nginx. 127.0.0.1 limita esta primera prueba al equipo local.

Probar desde el navegador

Abrir http://localhost:8080 y pulsar el botón. Se debe ver la página y el contador. No abrir index.html con doble clic para esta prueba: eso no demostraría que Nginx está sirviendo el archivo.

Resultado esperado: web-aula aparece con estado Up en docker ps. En Docker Desktop se puede ver en Containers; si no aparece, borrar filtros de búsqueda y compruebar si está activada la opción de mostrar solo contenedores en ejecución.

Qué se ha empaquetado

La imagen contiene Nginx, sus dependencias y una copia de la página. No contiene Docker Engine, la máquina virtual completa ni el estado del contador. La etiqueta stable-alpine puede cambiar con el tiempo: para transportar exactamente el mismo paquete construiremos una vez y guardaremos esa imagen.

4. Guardar la imagen para trasladarla

WINDOWS Y MAC. Ejecutar estos comandos en la misma carpeta del proyecto. No se necesita detener el contenedor.

Registrar la identidad de la imagen

```
docker image inspect aula-web:1.0 --format '{{.Id}}'
docker image inspect aula-web:1.0 --format '{{.Os}}/{{.Architecture}}'
```

Guardar el ID sha256 y la plataforma como evidencia. Más adelante se va a comparar esos valores en Ubuntu.

Crear el archivo TAR

```
docker image save --output aula-web-1.0.tar aula-web:1.0
```

Esperar a que vuelva el indicador de la terminal. El comando puede terminar sin mostrar mensajes. Ahora sí existe un archivo transportable con las capas y la configuración de la imagen.

Comprobar el archivo en Windows

```
Get-Item .\aula-web-1.0.tar
explorer .
```

Comprobar el archivo en Mac

```
ls -lh aula-web-1.0.tar
open .
```

El tamaño depende de la imagen base y la arquitectura. La carpeta actual determina dónde se guarda el archivo.

Por qué usamos save y load

docker save guarda una imagen; docker load la recupera. docker export exporta el sistema de archivos de un contenedor y no conserva la imagen de la misma manera. Para esta práctica se utiliza save y load.

El TAR no incluye volúmenes externos ni información que solo existe en memoria. Como la página ya está incorporada mediante COPY y no usa volúmenes, bastará copiar el TAR a Ubuntu. No se necesita subirlo a Docker Hub ni compartir las credenciales de una cuenta.

Punto de control

- La página responde en localhost:8080.
- Existe aula-web-1.0.tar en la carpeta del proyecto.
- Registrar el ID y la arquitectura de la imagen.

Mantener el contenedor de origen encendido mientras se prepara Ubuntu. Cada motor Docker administra sus propios contenedores: usar el mismo nombre web-aula en dos equipos distintos no genera conflicto.

5. Instalar Docker Engine en Ubuntu

UBUNTU. Esta sección supone una VM de laboratorio sin otra instalación de Docker. Si Docker ya funciona, pasar a la sección 6.

Instalar requisitos y registrar la clave

```
sudo apt update
sudo apt install -y ca-certificates curl
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc
```

Ejecutar una línea a la vez y no continuar si aparece un error. sudo solicita la contraseña de Ubuntu. Usar docker.asc, sin una s adicional.

Configurar el repositorio para la versión y procesador

Copiar todo el bloque, incluida la última línea EOF. Detecta automáticamente noble para Ubuntu 24.04 o resolute para Ubuntu 26.04, y la arquitectura real de la VM.

```
sudo tee /etc/apt/sources.list.d/docker.sources <<EOF
Types: deb
URIs: https://download.docker.com/linux/ubuntu
Suites: $(. /etc/os-release && echo "${UBUNTU_CODENAME:-$VERSION_CODENAME}")
Components: stable
Architectures: $(dpkg --print-architecture)
Signed-By: /etc/apt/keyrings/docker.asc
EOF
```

La clave permite verificar los paquetes; el repositorio indica de dónde descargarlos.

Instalar y verificar

```
sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin
sudo systemctl enable --now docker
sudo docker run --rm hello-world
```

Resultado esperado: Hello from Docker! La primera ejecución descarga una imagen de prueba. --rm elimina ese contenedor al terminar, pero conservar la imagen descargada.

Se seguirá usando sudo para los comandos Docker de Ubuntu. No es necesario modificar los permisos del socket ni agregar usuarios al grupo docker para completar la práctica.

Si apt muestra “Release file … is not valid yet”, corregir el reloj siguiendo la sección 9 antes de continuar. No desactivar las comprobaciones de fecha de apt.

6. Transferir el archivo a Ubuntu

Preparar Ubuntu y obtener sus datos

UBUNTU. Instala el servicio SSH que recibirá la copia del archivo:

```
sudo apt install -y openssh-server
sudo systemctl enable --now ssh
whoami
hostname -I
```

Anotar usuario y la IP de red de la VM. No uses 172.17.0.1, que suele ser el puente interno de Docker. En el caso del docente fueron msb y 192.168.1.90; cada estudiante debe usar sus propios valores. Si UFW está activo y bloquea SSH, autorizar OpenSSH; no desactivar el cortafuegos completo.

Copiar desde Windows o Mac

WINDOWS Y MAC. Regresa a la terminal del equipo de origen, dentro de practica-docker. Sustituye USUARIO e IP_UBUNTU en el siguiente comando antes de ejecutarlo:

```
scp aula-web-1.0.tar USUARIO@IP_UBUNTU:~/
```

scp copia por SSH. El destino ~/ representa la carpeta personal del usuario en Ubuntu. Escribir la contraseña de Ubuntu y esperar al 100 %. En Windows, si scp no se reconoce, instalar la característica opcional Cliente de OpenSSH y volver a abrir PowerShell.

Verificar la primera conexión

Si el equipo pregunta por la autenticidad del servidor, compara la huella SHA256 con la que obtienes en la consola de Ubuntu:

```
sudo ssh-keygen -lf /etc/ssh/ssh_host_ed25519_key.pub
```

Para una huella de tipo ED25519, si coincide con la del Mac o Windows, responder yes. Si el cliente muestra otro tipo de clave, comparar la clave pública de ese tipo en /etc/ssh.

Comprobar la recepción

UBUNTU. Ejecuta:

```
ls -lh ~/aula-web-1.0.tar
```

Resultado esperado: el archivo aparece con un tamaño distinto de cero. Si scp muestra timeout, revisa la IP y la conectividad de VMware; si muestra connection refused, comprobar SSH con sudo systemctl status ssh. Si muestra permission denied, verificar usuario y método de autenticación.

7. Cargar y ejecutar en Ubuntu

UBUNTU. A partir de aquí no se necesita reconstruir la imagen ni copiar index.html o Dockerfile.

Cargar la imagen recibida

```
sudo docker image load -i ~/aula-web-1.0.tar
```

Resultado esperado: Loaded image: aula-web:1.0. load importa la imagen al almacenamiento de Docker; todavía no crea un contenedor.

Comprobar que es la misma imagen

```
sudo docker image inspect aula-web:1.0 --format '{{.Id}}'
sudo docker image inspect aula-web:1.0 --format '{{.Os}}/{{.Architecture}}'
```

Compara el ID con el registrado en origen. Debe coincidir si trasladaron exactamente la misma imagen de una plataforma. La arquitectura debe ser compatible con Ubuntu.

Crear el contenedor en Ubuntu

```
sudo docker run --pull=never -d --name web-aula -p 8080:80 aula-web:1.0
sudo docker ps
```

--pull=never exige usar la imagen local, sin descargar otra. El estado debe ser Up. Esta vez se publica 8080 en las interfaces de Ubuntu para que el navegador de Windows o Mac pueda conectarse. La web estará accesible desde la red que alcance la VM; úsarla en la red de ésta práctica.

Probar primero dentro de Ubuntu

```
curl http://localhost:8080
```

Debe aparecer el HTML de la página. Ubuntu Server no necesita un entorno gráfico ni un navegador instalado para hacer esta comprobación.

Abrir la página desde Windows o Mac

En el navegador del equipo anfitrión abre http://IP_UBUNTU:8080, sustituyendo la IP. En el ejemplo del docente: http://192.168.1.90:8080. Comprobar el botón y el contador.

Dirección en el navegador del anfitrión	Servidor que atiende
http://localhost:8080	El contenedor del Windows o Mac, si continúa activo.
http://IP_UBUNTU:8080	El contenedor de Ubuntu en la máquina virtual.

Si curl funciona en Ubuntu pero la IP no abre desde el anfitrión, revisar la red de VMware y la publicación del puerto. No usar 127.0.0.1:8080:80 en Ubuntu para este acceso remoto. No es necesario abrir puertos en el router de internet.

8. Demostrar el traslado y cerrar la práctica

Demostrar que Ubuntu funciona por separado

WINDOWS Y MAC. Detén solamente el contenedor de origen:

```
docker stop web-aula
```

Abrir una ventana privada del navegador y solicita http://IP_UBUNTU:8080. La página debe seguir funcionando, porque la sirve Ubuntu. Una pestaña abierta antes de detener el servidor puede seguir mostrando contenido en caché; solicita la página de nuevo.

Detener y reanudar en Ubuntu

```
sudo docker stop web-aula
sudo docker start web-aula
```

Ejecutar stop para detener y start cuando quieras reanudar. start utiliza el contenedor existente. No vuelver a usar docker run con el mismo nombre mientras ese contenedor exista.

Consultar el estado y los registros

```
sudo docker ps -a
sudo docker logs web-aula
```

ps -a incluye contenedores detenidos. logs permite revisar mensajes de Nginx y solicitudes recibidas.

Eliminar solo los recursos de esta práctica

UBUNTU. Cuando ya no necesites el contenedor ni la imagen:

```
sudo docker stop web-aula
sudo docker rm web-aula
sudo docker image rm aula-web:1.0
```

WINDOWS Y MAC. Si también quieres limpiar el origen, usar los mismos comandos sin sudo:

```
docker stop web-aula
docker rm web-aula
docker image rm aula-web:1.0
```

rm elimina el contenedor; image rm elimina la referencia a la imagen si no está en uso. Se puede conservar el TAR para cargarlo otra vez o borrarlo manualmente. No usar limpiezas globales que eliminen proyectos ajenos.

Qué ocurrió durante toda la práctica

build creó el paquete;

run creó un proceso aislado a partir de él;

save escribió el paquete en disco;

scp transportó ese archivo;

load lo registró en otro motor Docker;

run inició una nueva instancia en Ubuntu.

No se trasladó un contenedor encendido: se trasladó su imagen y se creo otro contenedor.

9. Resolver los problemas más frecuentes

El reloj de Ubuntu está atrasado

Si apt indica “not valid yet”, ejecutar sudo timedatectl set-ntp true, esperar y revisar timedatectl. Si sigue sin sincronizar, comparar la hora UTC y realizar un ajuste temporal.

MAC para consultar UTC:

```
date -u '+%Y-%m-%d %H:%M:%S'
```

WINDOWS en PowerShell para consultar UTC:

```
(Get-Date).ToUniversalTime().ToString('yyyy-MM-dd HH:mm:ss')
```

UBUNTU. Mantener la zona UTC y reemplazar el marcador por la fecha y hora actual recién obtenida; no copiar la fecha de una captura antigua:

```
sudo timedatectl set-ntp false
sudo timedatectl set-timezone Etc/UTC
sudo timedatectl set-time 'AAAA-MM-DD HH:MM:SS'
sudo timedatectl set-ntp true
date -u
sudo apt update
```

Cambiar solamente la zona horaria no corrigir un reloj atrasado.

Versión o arquitectura incorrecta

Comprobar cat /etc/os-release y dpkg --print-architecture. Recrear el bloque docker.sources de la sección 5 para detectar ambos valores automáticamente. No mezclar noble con resolute ni amd64 con arm64 por copiar comandos de otro equipo.

Si la imagen y la VM tienen distintas arquitecturas, usar una VM compatible o construir explícitamente para el destino. Ejemplo en Windows o Mac para un destino Intel/AMD:

```
docker buildx build --platform linux/amd64 --load -t aula-web:1.0 .
```

Para un destino ARM sustituir linux/amd64 por linux/arm64. Después repitir save y load. Probar esa imagen en un origen de otra arquitectura puede requerir emulación; esta variante no hace que una VM ARM pueda arrancar nativamente en una PC Intel.

Otros mensajes

Puerto ocupado: cambia 8080 por 8081 en el lado izquierdo de -p y en la URL.

Nombre ocupado: usar docker start web-aula o eliminar el contenedor anterior antes de reemplazarlo.

TAR inexistente: revisar la carpeta actual y volver a ejecutar save.

Permission denied en Docker de Ubuntu: usar sudo.

Clave inexistente: el nombre correcto es docker.asc, no dockers.asc.

10. Evidencias y preguntas de aprendizaje

Entregables del estudiante

- Captura de la página funcionando en Windows o Mac y del comando docker ps.
- Captura del TAR creado y de la plataforma de la imagen.
- Captura de Hello from Docker en Ubuntu y de los datos de su versión y arquitectura.
- IDs de imagen coincidentes en origen y Ubuntu.
- Captura de docker ps en Ubuntu y de la página abierta mediante la IP de la VM.
- Explicación breve de por qué Ubuntu sigue sirviendo la página al detener el contenedor de origen.

Fuentes oficiales

Docker Desktop para Windows
https://docs.docker.com/desktop/setup/install/windows-install/

Docker Desktop para Mac
https://docs.docker.com/desktop/setup/install/mac-install/

Docker Engine para Ubuntu
https://docs.docker.com/engine/install/ubuntu/

Guardar y cargar imágenes
https://docs.docker.com/reference/cli/docker/image/save/
https://docs.docker.com/reference/cli/docker/image/load/

Plataformas y publicación de puertos
https://docs.docker.com/build/building/multi-platform/
https://docs.docker.com/engine/network/port-publishing/

OpenSSH en Ubuntu
https://ubuntu.com/server/docs/how-to/security/openssh-server/

Guía preparada el 18 de octubre de 2026.